

Comparative Analysis of a Hybrid Explainable AI Code Review System

Project: Static Analysis + CodeBERT/FAISS Retrieval + RAG + CodeLlama + Streamlit

Scope note. This comparison separates qualitative system capabilities from numerical evaluation results. The reviewed papers and the proposed prototype use different datasets, tasks, labels, and scoring procedures. Therefore, the numerical chart is a reported-metrics reference and not a direct head-to-head ranking.

1. Comparative Analysis Table

The table compares the documented scope of the selected approaches. “Not stated” means that the supplied material did not provide enough evidence to mark the feature as implemented.

Model / Study	Static Analysis	RAG / Retrieval	Code Review Generation	Explainable / Structured Output	Severity / Prioritization	Multi-language	Main Limitation / Difference
LLM + Static Analyzers (2025)	Yes	Partial	Yes	Not stated	Not stated	Not stated	Combines analyzer signals and LLM generation, but the supplied summary does not establish the same structured severity pipeline.
RepoAudit-style System	Not stated	Yes	Yes	Partial	Not stated	Not stated	Repository-level auditing is central; explicit language-specific static analysis and deterministic severity mapping are not established in the supplied summary.
Fine-tuned LLM Code Review	Not stated	Not stated	Yes	Partial	Not stated	Not stated	Requires task-specific review data and training; the proposed project uses pretrained local models rather than fine-tuning.
CodeReviewQA Benchmark	Not central	Not central	Evaluation	Evaluation	Not central	Not central	Primarily an evaluation benchmark, not a complete end-to-end application.
CodeAgent (2024)	Partial	Not central	Yes	Yes	Not stated	Not stated	Multi-agent coordination supports checking and revision but can increase workflow complexity and computational overhead.
Proposed Project	Yes	Yes	Yes	Yes	Yes	Yes	Small prototype dataset and 8-chunk retrieval corpus; larger matched experiments are required for broad claims.

Interpretation: the proposed system integrates all six listed functional areas in one application: language-specific static analysis, retrieval, review generation, structured/explainable output, severity mapping, and multilingual support.

2. Graphical Comparative Analysis

The following figures are intended to reproduce the style of the supplied reference document. The feature-coverage chart is qualitative. The numerical chart compares reported metrics from different tasks and datasets, so it must not be interpreted as proof that one model is universally better.

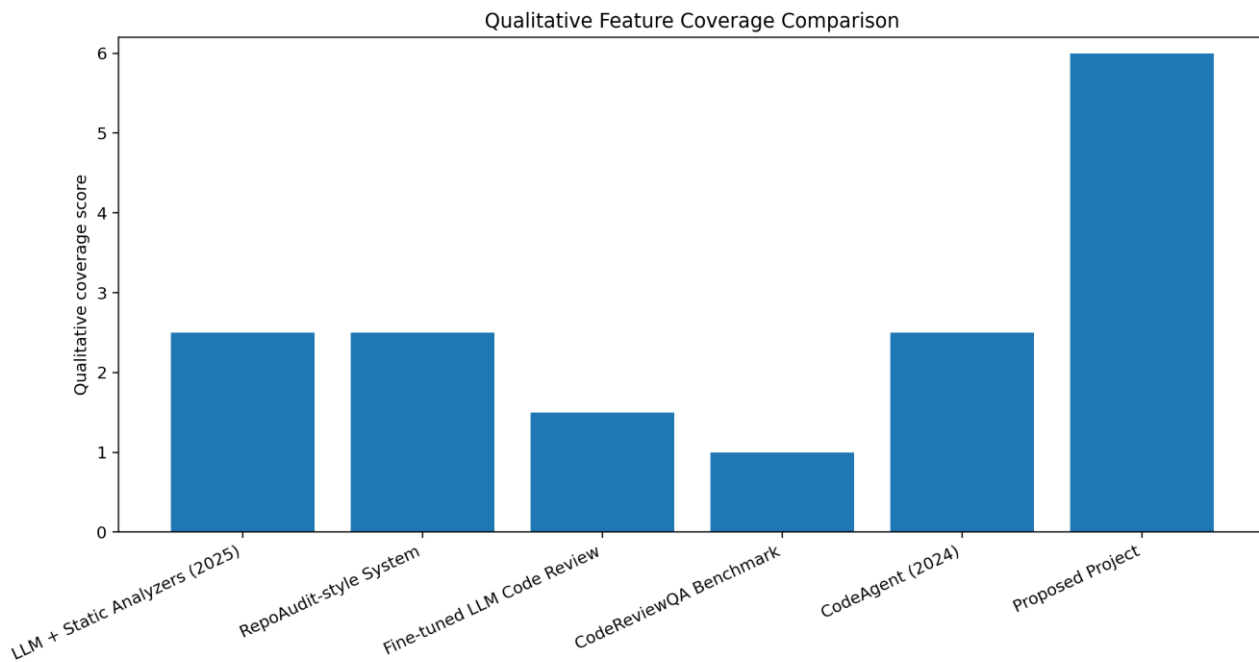


Figure 1. Qualitative feature-coverage comparison.

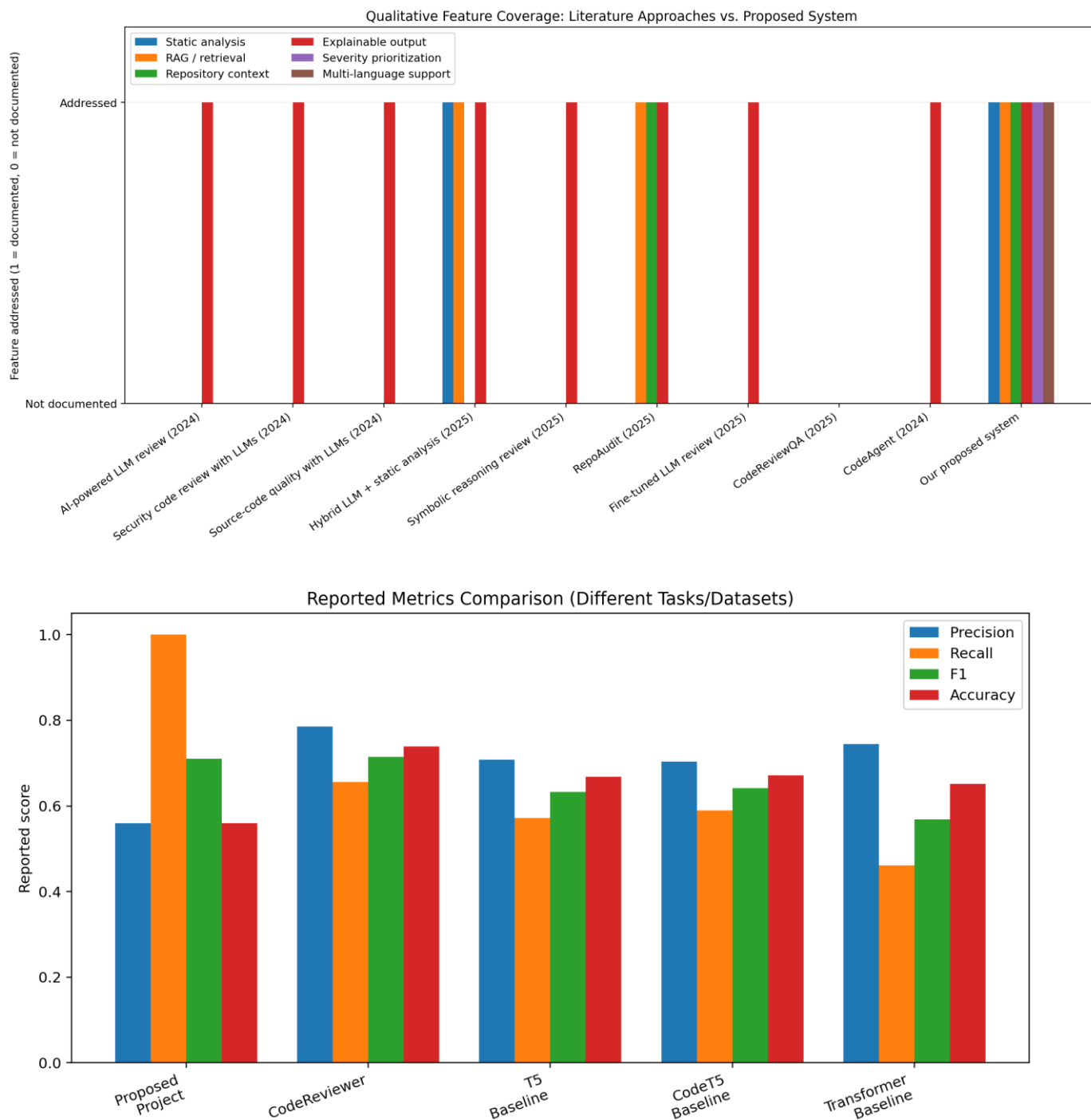


Figure 2. Reported metrics comparison with a task-and-dataset comparability warning.

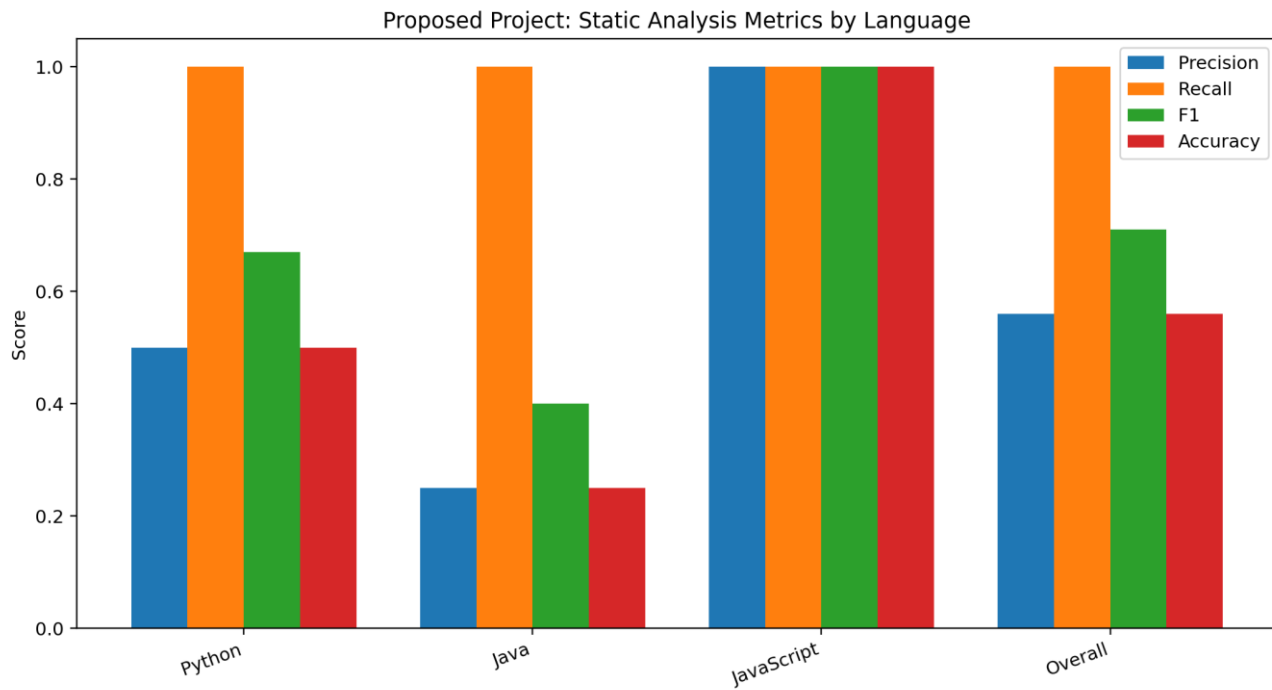


Figure 3. Static-analysis performance of the proposed project by programming language.

3. Numerical Comparative Analysis

The project's overall static-analysis results are compared with the CodeReviewer and baseline values reported in the supplied comparison document. The literature values concern code-change quality estimation, whereas the project values concern static analysis on a small multilingual prototype dataset.

Model / System	Task	Precision	Recall	F1	Accuracy
Proposed Project (overall static)	Small multilingual static-analysis dataset	0.56	1.00	0.71	0.56
CodeReviewer	Code-change quality estimation	0.786	0.656	0.715	0.739
T5 baseline	Code-change quality estimation	0.708	0.572	0.633	0.668
CodeT5 baseline	Code-change quality estimation	0.704	0.590	0.642	0.671
Transformer baseline	Code-change quality estimation	0.745	0.461	0.569	0.652

The proposed project recorded recall of 1.00 for the evaluated static findings, while precision was lower for Python and Java because additional findings were produced. JavaScript recorded 1.00 for all four reported static-analysis metrics in the prototype evaluation.

4. Proposed Project Evaluation Results

Language	Precision	Recall	F1	Accuracy
Python	0.50	1.00	0.67	0.50
Java	0.25	1.00	0.40	0.25
JavaScript	1.00	1.00	1.00	1.00
Overall	0.56	1.00	0.71	0.56

Retrieval Metric	Result	Interpretation
Recall@1	0.67	Two-thirds of evaluated retrieval cases succeeded at rank 1.
Recall@2	0.67	Two-thirds succeeded by rank 2.
Recall@3	1.00	All evaluated retrieval cases succeeded by rank

		3.
MRR	0.78	Mean reciprocal rank on the prototype retrieval corpus.
Review Quality Metric		Result
Precision		1.00
Recall		0.67
F1		0.80
Severity accuracy		1.00
Evidence grounding rate		1.00
Unsupported finding rate		0.00

5. Research Gap and Project Contribution

Observed Gap	Response in Proposed Project
Static analysis and LLM review may be treated separately	Combines Pylint, PMD, and ESLint findings with CodeLlama explanations in one pipeline.
Limited repository or code-context awareness	Uses CodeBERT embeddings and FAISS to retrieve relevant code chunks for the RAG prompt.
Generated comments can be difficult to validate	Uses structured output parsing with fields such as title, severity, evidence, and confidence.
Severity labels may vary between LLM outputs	Applies deterministic rule-based and semantic mappings after generation.
Single-language or limited language coverage	Routes Python, Java, and JavaScript to language-specific analyzers.
Inconsistent presentation of review results	Uses Streamlit to display a structured and explainable review report.

6. Comparative Discussion

- Existing approaches emphasize different priorities, including static-analysis integration, repository-level auditing, model fine-tuning, benchmark evaluation, or multi-agent review.
- The proposed project combines deterministic static analysis, semantic retrieval, RAG prompting, and LLM-based explanation in one end-to-end application.
- The output parser and severity classifier are implementation-level controls intended to make the generated review more consistent and easier to display.
- The current evaluation supports feasibility on the prototype dataset, but it should not be presented as proof of superiority over published systems without matched datasets, splits, labels, and metrics.
- The retrieval results are based on a small prototype corpus; larger and more diverse repositories are needed before making broad repository-level claims.

7. Conclusion

The main contribution of the proposed framework is the integration of language-specific static analysis, CodeBERT/FAISS semantic retrieval, RAG-based prompting, CodeLlama review generation, structured JSON parsing, and deterministic severity normalization within a single Streamlit application. The measured results demonstrate successful operation on the selected prototype evaluation set. Future work should include larger datasets, human assessment of review usefulness, ablation studies, latency and memory measurements, and matched benchmark experiments against published systems.

8. Reference Basis

- Combining Large Language Models with Static Analyzers for Code Review Generation (2025).
- RepoAudit: An Autonomous LLM-Agent for Repository-Level Code Auditing.
- Fine-Tuning Large Language Models to Improve Automated Code Review.
- CodeReviewQA: Code Review Comprehension Assessment for LLMs.
- CodeAgent: Autonomous Communicative Agents for Code Review (2024).

- Li et al. (2022), Automating Code Review Activities by Large-Scale Pre-training (CodeReviewer).
- Project workflow, evaluation report, and comparative-analysis documents supplied in the project workspace.

Prepared from the supplied project documents and evaluation results.